Industrial Experience Report

David Jarman
April 23rd, 2013
Senior Design I/II
SDSM&T

Innovative Systems LLC

January 2011 – May 2013

ManageMyVMail

January 2011 – August 2011, May 2012 – June 2012

The first project that I worked on for Innovative was the ManageMyVMail iPhone application. The project began with only two members including myself. We created a fairly stable prototype over the course of a summer and began working on a different project. When I came back to this app nearly a year later, I lead a new team that worked toward a released product.

The app would allow a user to login into their account and manage voicemails they had received. These functions included marking a message as read/unread, listening to the message, providing transcription of the message, and deleting the message.

ManageMyTV

August 2011 – May 2012, June 2012 – August 2012

After creating a prototype for ManageMyVMail, a new project was presented to my team. This new app would allow users to manage TV recordings on their home DVR as well as use their iPad as a remote control. The requirements were well defined for this project, allowing for more development and less research into best practices.

The development began in August 2011 and a prototype was requested to be available in October. My team and I worked hard and finally had a demo-able product available by the deadline. The project was continued until there was a stable version in Apple's app store.

Multiplayer Game Framework

August 2012 – May 2013

For my senior design project, I continued to work with Innovative Systems but with a different team. This project is an extension of a previous senior design project where single player games were built for Set-Top Boxes. An STB is a cable box that gets its television stream over the internet. Innovative has been working with this hardware and developing apps on them for a while.

Our goal was to overhaul the way games were being created and create an environment where multiplayer games could be played. The bulk of the project was spent creating a multiplayer server that could host communication between clients and also load plugins to provide game logic to the client. This was a drastic change from the previous year where games were compiled into the STB firmware. Now, there is a generic app installed on the STB that displays the game but performs no logic moves. Games are now developed as plugins that can be loaded into the server at runtime.

Appendix I : Coop report

Introduction

For the duration of the spring, summer, and fall of 2011, I worked at Innovative Systems doing iPhone and iPad application development. This report will explain the apps I helped create, the tools I used, and the experience I've gained by working at a software oriented company.

Background

When I started working for Innovative Systems in January 2011, my first project assigned was to learn the C# language and begin writing small applications with it. After about a month, I was assigned to the Apple team, where my task was to help design and implement an iOS app that would control a DVR. After several months of building the app for the iPhone and then the iPad, the Apple team was redirected toward making an app that would display voicemail and allow a user to listen to it. The voicemail app development continued until the end of the summer.

Since I was assigned to be working with mobile Apple devices, my C# training did me little good. Apple uses a language called Objective-C which is a hybrid language, much like C++, but with small-talk based syntax for objects and method calling. The framework that Apple uses for GUI development is called Cocoa and the IDE that Apple provides is called Xcode. Since all these tools are only available on the Mac OS X, I've been using a Mac Mini since joining the Apple team. My project partners and I use Mercurial to keep our code up to date.

For the first month, most of my time on the Apple team consisted of reading tutorials and writing small applications. I was able to use my knowledge of object-oriented programming that I learned simultaneously with C# to help design and implement classes in Objective-C.

APMax

Innovative Systems is mostly a software oriented company, but they do have one piece of hardware that was designed and is maintained within the company. This piece of hardware is called the APMax and is a revolutionary piece of equipment. It was designed specifically to be used by telephone companies for digital services such as call forwarding, caller ID, and voicemail; however, it also supports IPTV (internet protocol television).

Each app I helped make this summer was centered on interfacing with an APMax and then being able to push data to and pull data from it. In this way, we were able to easily access data we needed to make the apps work properly and usefully.

DVR Controller Requirements

When the Apple team was assigned to the DVR Controller app for iPhone/iPod Touch, a Senior Design group, working at Innovative Systems, had been working on a similar app for the Windows Phone 7 OS (WP7). Our task was to make an app that would have an Apple iOS look and feel and still have all the functionality of the WP7 app.

In order to achieve this same functionality, our app needed to have a nice looking guide that would allow the user to view TV listings, select, and then record them. Midway through the summer, the guide requirement was changed to a different look and feel so that the user had to select a channel, select a day, and that would bring up a table holding the listings for that channel for that day.

Additional required functions were to view the recordings that the user had selected, either from the phone or from their STB, control the users STB from their phone, search for listings, and change local settings for the app.

At the start of the project, the iPhone and iPad versions were to look similar in look and feel, but by the end of the co-op, the requirements were changed for the iPad version, and the Apple team began work revising the iPad version to meet its new requirements.

VoiceMail Requirements

Unlike the DVR Controller, the Apple team was the first of the next generation phone teams to begin work on the VoiceMail app. The requirements for this app were simple; all it needed to do was allow the user to listen to their voicemail, delete individual recordings, mark the recordings as “read”, and change a few basic settings for their voicemail service.

Midway through the co-op, an additional requirement was added that would allow the user to change their phones settings for a service called single number. This was given low priority and the Apple team did little work on it.

Project

DVR Controller

- Overview (version 1)

The DVR Controller app for iPhone/iPod Touch/iPad began with slow progress, as none of my team members, including myself, had any experience with Objective-C and the Cocoa Framework. There was very little communication between members of the team for the first month since we were all learning a new language in our own ways. Originally, there were three people on the Apple team: myself, Jarrett Brenner, and Daniel Heavlin. Dan was in his last semester and had a small class load, so he was able to put forth more time to the project than Jarrett and me. When I was ready to begin coding, Dan had laid down a good basis for the project. Progress that he had made included a fully functional guide (Figure 1).

My first contribution to this app was creating a custom table cell that displayed show information such as the show's name, the episode title, etc. Figure 2 shows how these custom cells looked in the context of the search page. Once these cells were created, I began work on the Program Search page and the logic that took place behind it. To perform searches, I had to make a static function that would take in a string that held the name of the search query and an enum that would be given an error type, if an error would arise during the search. The function returned an array, holding the results of the search. Once the search was complete, the user could scroll through the results and choose a show. Once a cell is selected, a details page is brought into view, giving more information on the selected show. The results could also be filtered by whether it was an HD show, new, or both. The "Sort by" button on the top right would bring up a menu prompting the user to select a sort type. The available options were "Date", "Name", and "Channel".

Figure 1

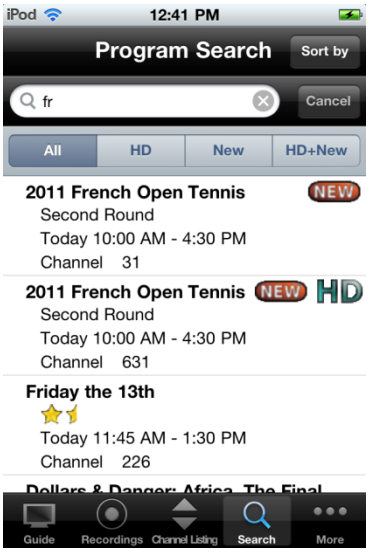

Figure 2 – Searching for "fr"

The second page I worked on was the Channel Listings. All this page did was show the user a list of all the channels. If they selected one of the channels, it would bring them to that particular channel on the guide page. There was also a favorites filter that would allow the user to only view their favorite channels.

When the app was near a versioning point, we moved our focus to the voicemail app, but came back to the DVR Controller app after about a month and a half. When we came back to the DVR app, we had new requirements and were ready to begin working on a new version of the

app that would have a more basic way of navigating the listing data, and would include a fully functioning virtual remote control.

DVR Controller (version 2)

The first major revision of the DVR Controller app that I was involved in was the complete renovation of the guide page. The user would now have to select a channel, and then select a day, and that would bring up a list of all the shows that would be aired on that particular day and on that particular channel. The functionality that was on the channel listing page was ported over to the guide page, meaning that the user would now be able to add favorite channels and filter current favorite channels. Another feature that was added was caching the listings to a database, to reduce the amount of service calls to the APMax.

The second major revision to the DVR Controller app was creating a virtual remote control that would allow the user to control their set top box directly from the app. I was put in charge of the user interface while my partner, Jarrett, worked on the service calls. Once both of us were done, we rebased our code, and I hooked up the GUI to the service calls so that pressing buttons would actually run code. One of the more challenging parts of creating the remote user interface was creating custom playback buttons. To do this, I used GIMP (GNU Image Manipulation Program) for Mac and created the buttons from scratch.

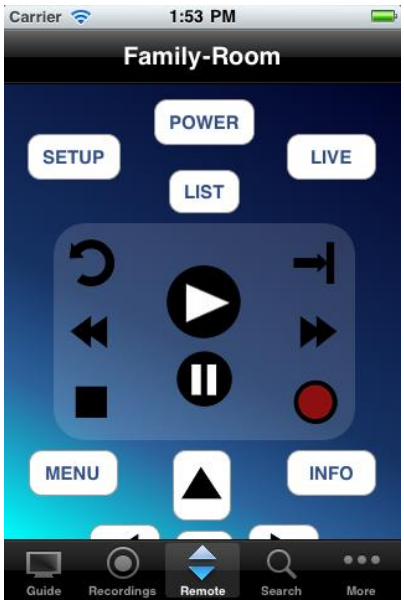

Figure 3 – Top half of the remote.

DVR Controller (Ultimate iPad App)

One thing to note about iOS development is that there are two platforms it umbrellas: the iPod/iPhone and the iPad. One option that developers can choose to utilize is to make their app look the same for both the iPad and iPod/iPhone. This requires much less effort but the iPad version just doesn't look right using all the same features as the iPod/iPhone version. The other option is to create a separate app for the iPad and take full advantage of its modified toolkit.

By version 2 of the DVR Controller, the Apple team had utilized the first option. In the last week of the summer, the CEO/President/Founder of Innovative Systems, Roger Musick, had designed what he called the "Ultimate iPad App". This app was to take full advantage of the iPad's larger display by displaying a bigger version the guide shown in Figure 1. This larger guide would take up the bottom 2/3rd of the screen, while on top, there would be some remote functionality, and info on shows currently being watched and recorded.

The major difference with this project was that we were given a deadline that we had to keep. We had to have a working version of the app so that it could be demonstrated at a tradeshow in Louisiana in mid-October. Considering that my partner, Jarrett, and I had only 7 months experience with Objective-C and only 2 months of in-depth iPad experience, this was a very tight deadline, not to mention the fact that both of us were back in school and able to put in 10-15 hours a week.

Even though the basic layout of the app was already made for us, there was much design left to do. Jarrett took care of the back end, such as service calls and database management, whereas I worked on GUI design and access to custom controls. One design pattern we decided

to implement was a factory: a singleton object that would act like a delegate to create and pass back custom control objects.

The timing of this project coincided with the class I took this Fall, CSC 421 – Graphical User Interfaces and Object Oriented Programming (GUI/OOP). This class, taught by Dr. John Weiss, taught me how to do custom drawing for a GUI component, which I eventually used in the drawing of dynamic buttons for the guide. My data structures knowledge also came in handy during this project. In order to make our code more efficient, I implemented a stack to hold onto previously allocated buttons, so that once several thousand buttons were made, no more needed to be allocated since they could be reused off the stack.

During the week before our deadline, Jarrett and I had to put in many more hours in order to accomplish our task. We finally did make a fairly stable and nice looking app for the company to demo in Louisiana.

VoiceMail

The VoiceMail app was the shortest project the Apple team worked on during the summer. We were given the requirements, which were very basic, and then we began. For the first week, Jarrett worked on the service that would download the audio files and then play them to the user while I worked on the interface for playing message and the settings. Figure 4 shows what the user would see when viewing their messages.

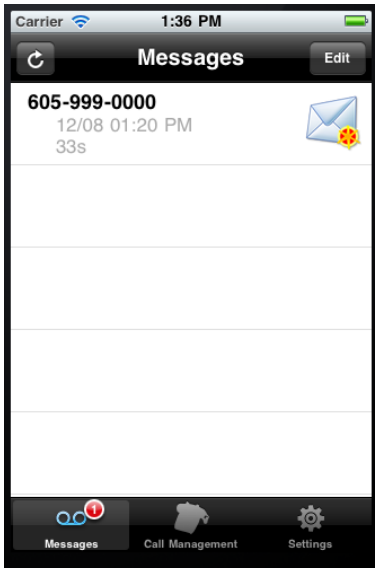

Figure 4 - Messages

Once the service code was completed, Jarrett and I rebased our code

and we hooked up the service calls to the interface. We tested the app for about a week making small changes here and there, and then demonstrated the app at Innovative System's main offices in Mitchell, SD, during a business trip over the summer.

Conclusion

During my coop with Innovative Systems, I gained much experience in dealing with partners, project deadlines, learning a new programming language, and overcoming disappointment. When I found out that I would be working with Apple devices, near the beginning of the coop, I was not very excited. But as it turns out, I now have a skill that helps separate myself from the other computer science majors, and that's being a multilingual programmer. I am now proficient in C/C++, C# and Objective-C, which helps me be a more flexible programmer. And besides just having learned new skills, I enjoyed working with Innovative Systems. It is an excellent company to work for that pushes its employees to be the best they can be.

Reflecting on the coop as a whole, it has helped me narrow down what I would like to do with my degree when I obtain it. Graphical User Interface programming is very frustrating and also very rewarding. At the end of the day, I have a product to show people that isn't a command prompt with a bunch of cryptic messages. Once the apps that I have helped develop are released, people around the nation will use my product and hopefully be happy with the experience I have helped create for them.